STABLECOİN CHECKOUT

Arcora

The customer pays with one signature. The merchant settles in the stablecoin they want, deterministically. On Arc.

`v1.1.0` · live on Arc testnet · `arcorapay.xyz`

The problem

Stablecoins moved **~$5T in 2025**. The holder-to-holder UX is fine. The merchant UX is not:

- The customer's chain is not the merchant's chain.
- The customer's token is not the merchant's token.
- Bridges, DEXes, approval clicks — each one is a place the customer abandons.
- The fallback is a custodial off-ramp that takes **days** and **1–2%**.

> *"I have USDC on Arbitrum. The merchant wants EURC on Arc. Five clicks and twenty minutes later I'm not sure if my money got there."*

What Arcora is

A Stripe-shaped checkout for stablecoin payments.

The merchant invoices in their preferred stable. The customer signs once — a Permit2 EIP-712 message, no transaction, no native gas. Arcora's relayer pulls the funds, runs the swap on Circle's App Kit, and deposits the merchant's stable into a 7-day custody escrow that the merchant claims permissionlessly.

> *Customer pays from where they are, with whatever they have.*
> *Merchant receives the stable they want, on Arc.*

No token. No custody. No FX gymnastics for the merchant.

Live, right now

Hosted checkout + dashboard: `arcorapay.xyz`

Docs: `docs.arcorapay.xyz`

npm: `npm install @arcora/sdk @arcora/sdk-react`

Gateway (V11, Arc testnet):

`0x07BAC123…aE3a3`

```
npm install @arcora/sdk
```

How it works — three touch points

01	Merchant creates an invoice	One API call (or one click in `/m/dashboard`)
02	Customer signs once at the checkout	Permit2 EIP-712 — no transaction, no gas
03	Relayer settles atomically	Permit2 pull → App Kit Swap → `settleInvoice` → custody escrow

```
import { Arcora } from "@arcora/sdk";

const arcora = new Arcora({ apiKey, environment: "testnet" });
const inv = await arcora.createInvoice({
  amountUsdc: 49.99,
  payInToken: "EURC",
  successUrl: "https://yoursite.com/orders/done",
});
arcora.openCheckout(inv);
```

What ships in v1.1

- **Permit2-based settlement** — customer signs **once**, no on-chain approve, no gas
- **Custody-escrow gateway (V11)** — funds sit in the contract, not the merchant wallet, until the 7-day refund window closes
- **Atomic FX via Circle App Kit Swap** — RFQ-priced USDC ⇄ EURC on Arc's maker network
- **Same-token fast path** — no swap when payIn = payout
- **Refund within window** — merchant or refund-delegate returns the customer's exact `amountOut`
- **Admin recovery** — abandoned-merchant escrows sweepable after 14d (refund + recovery windows)
- **Compliance gate** — config-flip Elliptic / TRM Labs hooks (Noop on testnet today)
- **HMAC-signed webhooks** — `invoice.paid` / `refunded` / `failed` / `claimed` with SSRF

Architecture

```
Customer wallet —(Permit2 EIP-712)——→ Arcora app  ——→ relayer queue
                                        (Vercel)        │
                                                        ▼
                                     ┌──── Relayer (VPS, Vault-backed key) ───┐
                                     │  pulls Permit2  →  App Kit Swap → settle│
                                     └──────────────────────┬──────────────────┘
                                                            ▼
                                          ArcFXGatewayV11 (Arc testnet)
                                               │
                                               ├─ escrow[globalId]  (7d refund window)
                                               ├─ protocolFeesAccrued
                                               └─ emit InvoicePaid + SettlementContext

Indexer (VPS)  ——→ reconciles chain → Neon Postgres
Webhooks (VPS) ——→ HMAC-sign + deliver to merchant endpoints
```

One repo. One team. One deploy graph. The contract is intentionally small — invoice lifecycle + escrow + fee accumulator

Arcora sits above Arc primitives

Arc itself ships first-party financial rails. Arcora is the **merchant abstraction layer** on top — the shape Stripe Checkout has on top of card networks.

Concern	Arc primitive	Arcora
FX swap on Arc	App Kit Swap	—
Crosschain USDC bridge	App Kit Bridge	—
Server-side wallets	Circle DCW	—
Compliance adapters	App Kit hooks	—
Invoice lifecycle + custody escrow	—	ArcFXGatewayV11
Hosted checkout + Permit2 flow	—	`/i/[invoiceId]`
Merchant dashboard + treasury	—	`/m/*`
Refund / claim primitives	—	**`refundInvoice`** · **`claim`**
TypeScript SDK + React + WordPress	—	`@arcora/sdk` family

Where we are

Surface	State
Contracts (Foundry)	**77 tests passing** — including a 256-run fuzz on protocol-fee invariant
App vitest suite	unit + route handlers for the full /api/* surface
SDK + React + WordPress plugin	published, in-tree, integrated
Live deployment	`arcorapay.xyz` aliased to Vercel, `arcora-shop.vercel.app` dogfooding the checkout
Custody-escrow contract	V11 live on Arc testnet (`0x07BAC123…aE3a3`)
Relayer + indexer + webhooks	running on a single host, Vault-isolated key

Pre-revenue. Testnet. No volume claims — Arc itself is testnet, and so are we.

Why Arc, why now

Arc is Circle's stablecoin-native L1, purpose-built for payment rails:

- **USDC as the gas token** — no native ETH friction for the merchant or the relayer
- **CCTP V2** for crosschain USDC, native to Arc
- **App Kit Swap + Bridge** as supported primitives, not third-party DEXes
- **Stablecoin-first economy** by design

Now because:

- Stablecoin payment volume crossed $50B/month in 2025
- Circle Gateway + CCTP V2 unlocked native unified balances across chains
- Arc testnet is open and stable; mainnet is on the horizon
- Which stablecoin checkout becomes the default on Arc is still up for grabs

The roadmap that defines the moat

v1.0 LIVE	Arc-only USDC / EURC	shipped 2026-04
v1.1 LIVE	Custody escrow, compliance hooks, Vault-isolated relayer key	shipped 2026-05
v1.x	Multi-stable (USDT, PYUSD, DAI, USDe) on Arc	in flight, mainnet-bound
v2.0	Crosschain USDC via Arc App Kit Bridge (Ethereum, Arbitrum, Optimism, Base, Polygon, Avalanche, Linea, Codex)	spec'd
v2.1	Source-side aggregator — any token on the source chain (native ETH, any ERC-20)	following
v2.2	Solana / Sui / non-EVM	following
v3.0	One-signature intent solver — full Stripe-like UX	endgame

v2.0 — crosschain via App Kit Bridge — is the differentiator. Each row builds on the

What stands between us and mainnet

1. **Arc Network mainnet launch** (out of our control — we follow)
2. **External audit** (Spearbit / Cantina / Sherlock RFP — trigger: first paying merchant or funding round)
3. **Multisig admin migration** (single EOA today → 2-of-3 or 3-of-5)
4. **KYB go-live** (`ManualKybProvider` + `PersonaProvider` adapters scaffolded)
5. **Vault hardening** (TLS listener, dedicated Unix user, HSM-isolated signer)
6. **Real Chainlink price feeds**
7. **Bug bounty** (Immunefi engagement with first paying merchant)

Each row is scoped. Items 5–6 are reversible single-host changes. Items 1–4 and 7 need external coordination.

Ask

For builders / partners:

Try the demo, integrate the SDK, file issues.

`npm install @arcora/sdk` — it works today.

GitHub: `Kubudak90/arc-fx-gateway`

For investors / Arc ecosystem:

v1 ships on testnet today. v2 (crosschain) is the wedge.

Looking for: ecosystem support on Arc mainnet, pre-seed for KYB + audit + branding, distribution partners.

For everyone:

The customer-side problem — **right token, wrong chain** — is real. Arcora's bet is one checkout, not five steps.

DEMO · Q&A

Pay anywhere. Settle on Arc.

`arcorapay.xyz` · `@arcora/sdk` · `arc-fx-gateway` (GitHub)

Hüseyin · 2026